SYNAPSE-CRS · LABORATORY OBSERVER-ONLY CONTROL PLANE

100 Viva Questions and Answers

Grounded in repository source, API routers, Alembic migrations, frontend routes, and research reports. In-memory hotpatching is **not implemented**. Laboratory envelopes are not /proc telemetry. SAT is not host safety.

API 0.1.0 · Contracts 1.0.0 · Confidentiality: no secrets · University viva pack

Synapse-CRS — 100 Viva Questions and Detailed Answers

Answers are grounded in this repository. If a capability is not implemented, the answer says so. Do not memorize invented endpoints. There is **no** GET `/api/v1/telemetry/events`.

Each item: **QUESTION**, **SHORT ANSWER** (20–30s), **DETAILED ANSWER**, **PROJECT-SPECIFIC IMPLEMENTATION**, **POSSIBLE FOLLOW-UP**, **BEST RESPONSE**.

- Synapse-CRS — 100 Viva Questions and Detailed Answers
- Project fundamentals
  - Q1. What is Synapse-CRS in one sentence?
  - Q2. Who is the intended user?
  - Q3. What does observer-only mean?
  - Q4. Which components are in the repo today?
  - Q5. What is DEMO MODE versus LAB ENVELOPES versus LIVE?
  - Q6. What license does the project use?
  - Q7. What versions should you quote?
  - Q8. What is fail-closed?
  - Q9. Where does the operator work?
  - Q10. What is the biggest one-line limitation?
- Problem statement and objectives
  - Q11. What problem are you solving?
  - Q12. Why not auto-block attackers?
  - Q13. List implemented objectives.
  - Q14. How is this different from a SIEM?
  - Q15. What is target authorization in this project?
  - Q16. What does capability telemetry.ingest mean?
  - Q17. Why a linear case machine?
  - Q18. What success looks like in a viva demo.
- System architecture
  - Q19. Why a modular monolith?
  - Q20. Walk a write request.
  - Q21. Health versus ready?
  - Q22. Why refuse MIGRATION_DATABASE_URL in the API?
  - Q23. What packages exist and why?
  - Q24. How does the agent fit?
  - Q25. Frontend architecture in three layers.
  - Q26. How is pagination done?
  - Q27. CORS policy.
  - Q28. Where are JSON Schemas?
  - Q29. What is identity_status enrolled?
  - Q30. How would it scale to thousands of agents?
- Frontend and backend
  - Q31. How does the console authenticate?
  - Q32. How are API errors shown?

    - Q33. Command palette actions?
    - Q34. How is the threat graph built?
    - Q35. Evidence Lab layout?
    - Q36. Why distinct workspace layouts?
    - Q37. FastAPI docs URL?
    - Q38. How are roles enforced?
    - Q39. What tests exist for the UI?
    - Q40. Request timeout?
- Database and API
    - Q41. Which migrations exist?
    - Q42. synapse_app versus owner?
    - Q43. How are UUIDs used?
    - Q44. Describe TargetCreate validation.
    - Q45. Case transition request?
    - Q46. List GET versus POST ingest.
    - Q47. Audit verify response?
    - Q48. Correlation run body?
    - Q49. Reasoning run body?
    - Q50. What is 409 used for?
- Cybersecurity and telemetry
    - Q51. Which event types are allowed?
    - Q52. What is boot_id for?
    - Q53. How is redaction tested?
    - Q54. Enrollment versus monitoring.
    - Q55. What stops a random key from ingesting?
    - Q56. Can viewers ingest?
    - Q57. Process identity tuple?
    - Q58. Finding types names.
    - Q59. Epistemic correlation versus inference.
    - Q60. Risk contribution versus confidence.
    - Q61. What is content_class / basis on envelopes?
    - Q62. Laboratory envelopes vs genuine /proc.
- Digital signatures and audit integrity
    - Q63. How does Ed25519 work here?
    - Q64. Why a domain string?
    - Q65. Canonical JSON rules?
    - Q66. Audit hash formula?
    - Q67. What if an attacker UPDATEs an audit row as synapse_app?
    - Q68. What does verify do?
    - Q69. Are tokens hashed at rest in .env?
    - Q70. TLS?
- Neuro-symbolic reasoning and algorithms
    - Q71. What does neuro-symbolic mean here?
    - Q72. Why Isolation Forest?
    - Q73. What is unusualness versus probability?
    - Q74. How is H4 selected?
    - Q75. Name H1 H2 H3 H4.

    - Q76. Feature ablation / counterfactual?
    - Q77. Event-count prediction?
    - Q78. Z3 SAT meaning?
    - Q79. Write the policy in words.
    - Q80. Risk formula again with numbers.
    - Q81. Reconstruction completeness?
    - Q82. Does the system do causal inference?
- Testing, datasets and research methodology
    - Q83. How is the Python suite run?
    - Q84. What does Phase 10 measure?
    - Q85. Phase 11?
    - Q86. Phase 12 and 13?
    - Q87. Why fail-closed on missing data?
    - Q88. Frontend tests you can name.
    - Q89. What is dataset_label synthetic_lab?
    - Q90. Reproducibility commands?
- Limitations, deployment and future scope
    - Q91. Does the platform perform in-memory hotpatching?
    - Q92. Can it automatically block an attacker?
    - Q93. Biggest technical limitation?
    - Q94. How do you deploy?
    - Q95. Secrets management?
    - Q96. What if the database goes down?
    - Q97. OIDC future?
    - Q98. How would you add real /proc telemetry honestly?
    - Q99. What must you not say in the viva?
    - Q100. What is your personal contribution story (template)?

Project fundamentals

Q1. What is Synapse-CRS in one sentence?

QUESTION: What is Synapse-CRS in one sentence?

SHORT ANSWER: A laboratory observer-only control plane and React console for authorized targets, signed telemetry, correlation, hybrid reasoning, and audit — not a hotpatching engine.

DETAILED ANSWER: The repository implements FastAPI + PostgreSQL + packages for policy, cases, audit, telemetry, correlation, reasoning, and a Vite console. The marketing name includes in-memory hotpatching; that module does not exist.

PROJECT-SPECIFIC IMPLEMENTATION: README.md and docs/safety/claims.md; API_VERSION 0.1.0 in packages/contracts.

POSSIBLE FOLLOW-UP: Then why keep the hotpatching phrase in the title?

BEST RESPONSE: It is the research programme name. In a viva I must separate aspiration from tests. I will not claim a patcher.

Q2. Who is the intended user?

QUESTION: Who is the intended user?

SHORT ANSWER: A lab operator or student with an admin or viewer bearer token, on owned lab/development assets.

DETAILED ANSWER: Scope environment is only lab or development. There is no multi-tenant IdP.

PROJECT-SPECIFIC IMPLEMENTATION: TargetScope in packages/contracts/scope.py; SYNAPSE_AUTH_TOKENS in config.py.

POSSIBLE FOLLOW-UP: Can the public internet use it?

BEST RESPONSE: Not as designed. Tokens are shared secrets (ADR 0002), CORS is an allowlist, and claims forbid unauthorized targets.

Q3. What does observer-only mean?

QUESTION: What does observer-only mean?

SHORT ANSWER: The API stores state and evidence; it never starts a program on a target. ACTION still has executed=false.

DETAILED ANSWER: docs/decisions/0006-no-execution.md. No command route, no shell, no firewall API.

PROJECT-SPECIFIC IMPLEMENTATION: CaseTransitionView.executed Field description in contracts/cases.py.

POSSIBLE FOLLOW-UP: Is VERIFICATION a formal proof?

BEST RESPONSE: No. It is a case-state label. Z3 runs on recommendation records, not on that label alone.

Q4. Which components are in the repo today?

QUESTION: Which components are in the repo today?

SHORT ANSWER: Control plane, Postgres schema, Linux procfs agent package, correlation, hybrid reasoning, Z3 policy checks, research harnesses, React console.

DETAILED ANSWER: Roadmap.md is stale if read as 'nothing after v0.1'. Inspect packages/ and frontend/ instead.

PROJECT-SPECIFIC IMPLEMENTATION: apps/api, packages/*, frontend/src, tests/.

POSSIBLE FOLLOW-UP: Is the Linux agent required for the UI demo?

BEST RESPONSE: No. The UI can show DEMO MODE fixtures or laboratory envelopes ingested through the real POST ingest path.

Q5. What is DEMO MODE versus LAB ENVELOPES versus LIVE?

QUESTION: What is DEMO MODE versus LAB ENVELOPES versus LIVE?

SHORT ANSWER: DEMO MODE is browser fixtures. LIVE is API rows. LAB ENVELOPES is live API with dataset_label synthetic_lab — signed constructed events, not /proc.

DETAILED ANSWER: frontend/src/lib/provenance.ts and data/demo.ts. usePlatform does not substitute demo on live failure.

PROJECT-SPECIFIC IMPLEMENTATION: provenanceFlag(); HUD in Shell.tsx.

POSSIBLE FOLLOW-UP: If live fails, do you see demo hosts?

BEST RESPONSE: No. Source becomes unavailable with an error; empty lists.

Q6. What license does the project use?

QUESTION: What license does the project use?

SHORT ANSWER: README states no license has been selected.

DETAILED ANSWER: Do not invent Apache-2.0 in a viva.

PROJECT-SPECIFIC IMPLEMENTATION: README.md License section.

POSSIBLE FOLLOW-UP: Can we open-source it tomorrow?

BEST RESPONSE: Only after the authors choose a license; the tree currently has none.

Q7. What versions should you quote?

QUESTION: What versions should you quote?

SHORT ANSWER: API 0.1.0, contracts 1.0.0, reasoning engine 0.6.0, frontend package 0.1.0.

DETAILED ANSWER: Quote code constants, not slide art.

PROJECT-SPECIFIC IMPLEMENTATION: synapse_contracts.version; frontend/package.json; synapse_reasoning ENGINE_VERSION.

POSSIBLE FOLLOW-UP: Is this production 1.0?

BEST RESPONSE: No. API 0.1.0 foundation plus research packages.

Q8. What is fail-closed?

QUESTION: What is fail-closed?

SHORT ANSWER: Missing config, missing corpora, or failed eligibility stops the action instead of guessing.

DETAILED ANSWER: Settings refuse empty tokens; Phase 11/12/13 exit without inventing CIC-IDS/DMBD scores.

PROJECT-SPECIFIC IMPLEMENTATION: config.py; packages/validation phase11/12/13 run.py.

POSSIBLE FOLLOW-UP: Is fail-closed the same as fail-secure in NCSC language?

BEST RESPONSE: Related idea: we refuse to proceed; we do not claim Common Criteria fail-secure certification.

Q9. Where does the operator work?

QUESTION: Where does the operator work?

SHORT ANSWER: Browser on http://127.0.0.1:5173 talking to http://127.0.0.1:8000.

DETAILED ANSWER: VITE_API_BASE_URL default. CORS must list that origin.

PROJECT-SPECIFIC IMPLEMENTATION: frontend/src/api/client.ts API_BASE; config cors_origins().

POSSIBLE FOLLOW-UP: Can you use https://localhost?

BEST RESPONSE: Only if that exact origin is in SYNAPSE_CORS_ORIGINS; wildcards are rejected.

Q10. What is the biggest one-line limitation?

QUESTION: What is the biggest one-line limitation?

SHORT ANSWER: No host execution and no independent dataset completion; synthetic n=12 is not enterprise detection.

DETAILED ANSWER: Pair engineering completeness with scientific incompleteness.

PROJECT-SPECIFIC IMPLEMENTATION: claims.md research evaluation limitation; benchmark-results.md.

POSSIBLE FOLLOW-UP: So the project failed?

BEST RESPONSE: The engineering pipeline exists; the scientific generalization bar was not met. That is a valid research outcome.

Problem statement and objectives

Q11. What problem are you solving?

QUESTION: What problem are you solving?

SHORT ANSWER: Investigations need enrolled, signed evidence and conservative hypotheses instead of spoofable events and overconfident scores.

DETAILED ANSWER: Spoofing, PID-only joins, ML-as-probability, SAT-as-safety, mutable audit tables.

PROJECT-SPECIFIC IMPLEMENTATION: telemetry-signing.md; evidence-correlation.md; scoring.py comments; verification.py explanation strings.

POSSIBLE FOLLOW-UP: Is this a new problem in literature?

BEST RESPONSE: Not unique, but the combination of fail-closed research reporting plus no-execution is the project stance.

Q12. Why not auto-block attackers?

QUESTION: Why not auto-block attackers?

SHORT ANSWER: v0.1 forbids execution. Wrong blocks on incomplete evidence are worse than H4.

DETAILED ANSWER: Decision kind execute_host_mitigation is forbidden in Z3 policy.

PROJECT-SPECIFIC IMPLEMENTATION: verification.py CONSTRAINTS mitigation_kind_forbidden.

POSSIBLE FOLLOW-UP: Isn't that useless for defense?

BEST RESPONSE: It is a research observer. Mitigation is a later catalogue with human approval in the roadmap, not this tree's executor.

Q13. List implemented objectives.

QUESTION: List implemented objectives.

SHORT ANSWER: Registry, cases, audit chain, signed ingest, correlation, hybrid reasoning, honest UI, fail-closed eval harnesses.

DETAILED ANSWER: Map each to a package and a test module.

PROJECT-SPECIFIC IMPLEMENTATION: tests/integration/* ; frontend vitest.

POSSIBLE FOLLOW-UP: Which objective is incomplete?

BEST RESPONSE: Independent evaluation on CIC-IDS2017 and DMBD process trees.

Q14. How is this different from a SIEM?

QUESTION: How is this different from a SIEM?

SHORT ANSWER: SIEM indexes many sources; we require enrollment+signatures, refuse telemetry list GET, and never execute playbooks.

DETAILED ANSWER: We also expose H4 and Z3 limitations in the UI.

PROJECT-SPECIFIC IMPLEMENTATION: No GET telemetry/events route; DefenceLab copy.

POSSIBLE FOLLOW-UP: Could it feed a SIEM?

BEST RESPONSE: Findings JSON could, but we did not build exporters.

Q15. What is target authorization in this project?

QUESTION: What is target authorization in this project?

SHORT ANSWER: An operator assertion: owner, scope, expiry, capabilities. It does not prove legal ownership.

DETAILED ANSWER: Eligibility fails if revoked or expiry not in the future.

PROJECT-SPECIFIC IMPLEMENTATION: packages/policy; TargetCreate contracts.

POSSIBLE FOLLOW-UP: Can I register 8.8.8.8?

BEST RESPONSE: Assets have a pattern and forbidden wildcards; environment must be lab or development — not a licence to scan the internet.

Q16. What does capability telemetry.ingest mean?

QUESTION: What does capability telemetry.ingest mean?

SHORT ANSWER: A reserved name on the target allowlist. It does not by itself collect packets.

DETAILED ANSWER: Capabilities are a closed enum of four strings.

PROJECT-SPECIFIC IMPLEMENTATION: packages/contracts/capabilities.py.

POSSIBLE FOLLOW-UP: Does checking the box start the agent?

BEST RESPONSE: No. An admin must enroll a key and a collector must be run separately.

Q17. Why a linear case machine?

QUESTION: Why a linear case machine?

SHORT ANSWER: ADR 0004: one legal next state, auditable, no silent skip to ACTION.

DETAILED ANSWER: SEALED_AUDIT is terminal. No backward edges in v0.1.

PROJECT-SPECIFIC IMPLEMENTATION: packages/cases; routers/cases.py.

POSSIBLE FOLLOW-UP: How do you abort a case?

BEST RESPONSE: You cannot in v0.1 except by not transitioning. That is a documented limitation.

Q18. What success looks like in a viva demo.

QUESTION: What success looks like in a viva demo.

SHORT ANSWER: Health/ready, live snapshot, labelled lab envelopes, H4 honesty, audit valid=true, no hotpatch claim.

DETAILED ANSWER: DEMO_RUNBOOK.md sequence.

PROJECT-SPECIFIC IMPLEMENTATION: DEMO_RUNBOOK.md; frontend HUD.

POSSIBLE FOLLOW-UP: What if findings are empty?

BEST RESPONSE: Say no correlation run or no enrolled events — do not invent a GET dump.

System architecture

Q19. Why a modular monolith?

QUESTION: Why a modular monolith?

SHORT ANSWER: One process, transactional writes with audit; packages stay unit-testable without HTTP.

DETAILED ANSWER: overview.md composition root.

PROJECT-SPECIFIC IMPLEMENTATION: synapse_api.main.create_app include_router list.

POSSIBLE FOLLOW-UP: When would you split services?

BEST RESPONSE: If ingest volume required a separate worker; not needed for the lab.

Q20. Walk a write request.

QUESTION: Walk a write request.

SHORT ANSWER: CORS and size middleware → bearer principal → admin check → service → SQLAlchemy commit with audit hash → JSON view.

DETAILED ANSWER: Eligibility 409 leaves state unchanged.

PROJECT-SPECIFIC IMPLEMENTATION: RequestGuardMiddleware; get_principal; audit_log.

POSSIBLE FOLLOW-UP: Is the UI part of that path?

BEST RESPONSE: Only as an HTTP client. Authorization is repeated on every API call.

Q21. Health versus ready?

QUESTION: Health versus ready?

SHORT ANSWER: Health is process liveness without DB. Ready executes SELECT 1 and may 503.

DETAILED ANSWER: Console lamps API vs DB use these two.

PROJECT-SPECIFIC IMPLEMENTATION: routers/health.py.

POSSIBLE FOLLOW-UP: Can health be ok and ready fail?

BEST RESPONSE: Yes — uvicorn up, Postgres down.

Q22. Why refuse MIGRATION_DATABASE_URL in the API?

QUESTION: Why refuse MIGRATION_DATABASE_URL in the API?

SHORT ANSWER: So the serving role cannot migrate or hold owner credentials.

DETAILED ANSWER: Compose injects that URL only into the migrate service.

PROJECT-SPECIFIC IMPLEMENTATION: main.py _reject_migration_credentials.

POSSIBLE FOLLOW-UP: How do you migrate then?

BEST RESPONSE: make migrate / Alembic with owner DSN in a separate process.

Q23. What packages exist and why?

QUESTION: What packages exist and why?

SHORT ANSWER: contracts, policy, cases, audit, telemetry, agent, correlation, reasoning, validation.

DETAILED ANSWER: ADR 0001: cases separate from policy so the state machine is pure.

PROJECT-SPECIFIC IMPLEMENTATION: docs/decisions/0001-package-layout.md.

POSSIBLE FOLLOW-UP: Does validation write telemetry_events?

BEST RESPONSE: No. Research harness is offline; it must not mix benchmark rows into enrolled evidence.

Q24. How does the agent fit?

QUESTION: How does the agent fit?

SHORT ANSWER: packages/agent polls procfs, signs envelopes, POSTs ingest. Optional for UI.

DETAILED ANSWER: Linux-only collector; queue bounds in architecture/linux-agent.md.

PROJECT-SPECIFIC IMPLEMENTATION: packages/agent/src/synapse_agent.

POSSIBLE FOLLOW-UP: Did the graded live demo use /proc?

BEST RESPONSE: The documented live seed used constructed envelopes with dataset_label synthetic_lab. Say that.

Q25. Frontend architecture in three layers.

QUESTION: Frontend architecture in three layers.

SHORT ANSWER: Shell+HUD, workspace pages, api/client + usePlatform snapshot.

DETAILED ANSWER: Threat Graph is lazy-loaded. ErrorBoundary wraps workspaces.

PROJECT-SPECIFIC IMPLEMENTATION: App.tsx; hooks/usePlatform.ts.

POSSIBLE FOLLOW-UP: Does the snapshot poll WebSockets?

BEST RESPONSE: No. Manual refresh; no WebSocket server.

Q26. How is pagination done?

QUESTION: How is pagination done?

SHORT ANSWER: limit default 50 max 100, offset ≥0, Page models with total.

DETAILED ANSWER: Console listAll currently uses the first page only (limit 100).

PROJECT-SPECIFIC IMPLEMENTATION: Query params on list routes.

POSSIBLE FOLLOW-UP: What if there are 10k audit events?

BEST RESPONSE: UI would show the first page; that is a limitation I should state.

Q27. CORS policy.

QUESTION: CORS policy.

SHORT ANSWER: Comma-separated exact origins; wildcard rejected at startup.

DETAILED ANSWER: Methods GET, POST; header Authorization, Content-Type.

PROJECT-SPECIFIC IMPLEMENTATION: create_app CORSMiddleware; Settings.cors_origins.

POSSIBLE FOLLOW-UP: Why credentials false?

BEST RESPONSE: Bearer header, not cookies.

Q28. Where are JSON Schemas?

QUESTION: Where are JSON Schemas?

SHORT ANSWER: schemas/*.schema.json version 1.0.0; Pydantic is runtime.

DETAILED ANSWER: Tests compare examples to schemas.

PROJECT-SPECIFIC IMPLEMENTATION: schemas/ and packages/contracts.

POSSIBLE FOLLOW-UP: Do you generate code from schema?

BEST RESPONSE: Hand-written Pydantic kept in sync by tests, not a codegen step.

Q29. What is identity_status enrolled?

QUESTION: What is identity_status enrolled?

SHORT ANSWER: Telemetry rows that passed enrollment+signature; correlation reads only those, max 5000 newest.

DETAILED ANSWER: Prevents mixing unverified rows into findings.

PROJECT-SPECIFIC IMPLEMENTATION: docs/architecture/evidence-correlation.md Storage.

POSSIBLE FOLLOW-UP: Could you correlate demo fixtures?

BEST RESPONSE: Browser demo never hits correlation; live demo ingested enrolled envelopes.

Q30. How would it scale to thousands of agents?

QUESTION: How would it scale to thousands of agents?

SHORT ANSWER: Not designed to. Single process, first-page UI, 5000-row correlation cap.

DETAILED ANSWER: Honest answer: shard ingest, async workers, cursor pagination — future work.

PROJECT-SPECIFIC IMPLEMENTATION: correlation engine cap in architecture doc.

POSSIBLE FOLLOW-UP: Any load test numbers?

BEST RESPONSE: No published k-agent benchmark in the repo. Do not invent QPS.

Frontend and backend

Q31. How does the console authenticate?

QUESTION: How does the console authenticate?

SHORT ANSWER: Operator pastes bearer into /session; sessionStorage; Authorization header on fetches.

DETAILED ANSWER: Token minLength 16; maskToken shows last 4 only.

PROJECT-SPECIFIC IMPLEMENTATION: SignIn.tsx; auth/session.ts; api/client.ts.

POSSIBLE FOLLOW-UP: localStorage?

BEST RESPONSE: No, sessionStorage so a tab close drops the token.

Q32. How are API errors shown?

QUESTION: How are API errors shown?

SHORT ANSWER: Client reads nested error.code/message, maps 401/403/404/422, never logs the bearer.

DETAILED ANSWER: Live 401 sets auth rejected and empty snapshot.

PROJECT-SPECIFIC IMPLEMENTATION: client.ts sanitizeBody; usePlatform loadLive catch.

POSSIBLE FOLLOW-UP: What did you fix in commit 2675b6d?

BEST RESPONSE: Parsers now understand {error:{...}} instead of top-level code.

Q33. Command palette actions?

QUESTION: Command palette actions?

SHORT ANSWER: Ctrl/Cmd+K: navigate routes, jump to loaded targets/cases, refresh, enter demo. No host commands.

DETAILED ANSWER: CommandPalette.tsx.

PROJECT-SPECIFIC IMPLEMENTATION: That file.

POSSIBLE FOLLOW-UP: Can it enroll an agent?

BEST RESPONSE: No such action; enrollment is admin HTTP, not palette.

Q34. How is the threat graph built?

QUESTION: How is the threat graph built?

SHORT ANSWER: Client-side from snapshot via graphFromSnapshot; dagre LR; custom kind nodes.

DETAILED ANSWER: No graph REST resource.

PROJECT-SPECIFIC IMPLEMENTATION: lib/graphFromSnapshot.ts; graphLayout.ts.

POSSIBLE FOLLOW-UP: Will 10k nodes work?

BEST RESPONSE: Not tested as a product; we memoize nodes and lazy-load the canvas.

Q35. Evidence Lab layout?

QUESTION: Evidence Lab layout?

SHORT ANSWER: Explorer, relation canvas, inspector, storyline. Filter box. Deep link ?finding=.

DETAILED ANSWER: Classifications observed/correlated/inferred from timeline payload.

PROJECT-SPECIFIC IMPLEMENTATION: EvidenceLab.tsx; Timeline.tsx.

POSSIBLE FOLLOW-UP: Process tree from Windows ETW?

BEST RESPONSE: No. We refuse invented process trees.

Q36. Why distinct workspace layouts?

QUESTION: Why distinct workspace layouts?

SHORT ANSWER: To avoid a generic card grid; Command is spatial; Graph is full-bleed; Reasoning is a vertical stack.

DETAILED ANSWER: styles.css grid areas.

PROJECT-SPECIFIC IMPLEMENTATION: frontend/src/styles.css; pages/*.

POSSIBLE FOLLOW-UP: Is that just cosmetics?

BEST RESPONSE: It encodes investigation workflow: overview → lock → evidence → relations → explanation.

Q37. FastAPI docs URL?

QUESTION: FastAPI docs URL?

SHORT ANSWER: /docs and /redoc unless SYNAPSE_ENVIRONMENT=production.

DETAILED ANSWER: OpenAPI still describes real routes.

PROJECT-SPECIFIC IMPLEMENTATION: main.py docs_url ternary.

POSSIBLE FOLLOW-UP: Should production expose /docs?

BEST RESPONSE: This project turns them off in production environment.

Q38. How are roles enforced?

QUESTION: How are roles enforced?

SHORT ANSWER: require_admin on write routes; get_principal on reads. Frontend hiding is not enforcement.

DETAILED ANSWER: Viewer POST target → 403.

PROJECT-SPECIFIC IMPLEMENTATION: auth.py require_admin; routers/targets.py post_target.

POSSIBLE FOLLOW-UP: Can I forge admin in React?

BEST RESPONSE: You can draw a button; the API still 403s.

Q39. What tests exist for the UI?

QUESTION: What tests exist for the UI?

SHORT ANSWER: Vitest: format, research catalog honesty, apiRequest, graphFromSnapshot, unique events, provenance, H4 copy. Not Cypress.

DETAILED ANSWER: npm test; npm run build runs tsc.

PROJECT-SPECIFIC IMPLEMENTATION: frontend/src/**/*.test.ts.

POSSIBLE FOLLOW-UP: Did you E2E the browser?

BEST RESPONSE: Manual browser QA against live API was done in development; automated E2E is not in package.json.

Q40. Request timeout?

QUESTION: Request timeout?

SHORT ANSWER: 12 seconds AbortController in apiRequest.

DETAILED ANSWER: AbortError becomes code aborted.

PROJECT-SPECIFIC IMPLEMENTATION: client.ts DEFAULT_TIMEOUT_MS.

POSSIBLE FOLLOW-UP: Retries?

BEST RESPONSE: No automatic retry loop; Refresh is manual.

Database and API

Q41. Which migrations exist?

QUESTION: Which migrations exist?

SHORT ANSWER: 0001 initial targets/cases/audit; 0002 enforce state; 0003 telemetry; 0004 agents; 0005 correlation; 0006 hybrid reasoning.

DETAILED ANSWER: Alembic env reads MIGRATION_DATABASE_URL or DATABASE_URL from .env file.

PROJECT-SPECIFIC IMPLEMENTATION: apps/api/alembic/versions/.

POSSIBLE FOLLOW-UP: Can I edit alembic.ini URL?

BEST RESPONSE: Placeholder only; runtime URL comes from environment.

Q42. synapse_app versus owner?

QUESTION: synapse_app versus owner?

SHORT ANSWER: Owner migrates and creates role. App connects with GRANT SELECT/INSERT and limited UPDATE. No DELETE on evidence/audit.

DETAILED ANSWER: init-roles.sh creates synapse_app on empty volume only.

PROJECT-SPECIFIC IMPLEMENTATION: infra/postgres/init-roles.sh; GRANT statements in revisions.

POSSIBLE FOLLOW-UP: Password rotation of synapse_app?

BEST RESPONSE: Must match DATABASE_URL; documented as a local ops concern, not a product UI.

Q43. How are UUIDs used?

QUESTION: How are UUIDs used?

SHORT ANSWER: Primary keys for targets, cases, agents, events, findings, assessments.

DETAILED ANSWER: Path params are UUID types in FastAPI.

PROJECT-SPECIFIC IMPLEMENTATION: routers/*.py UUID parameters.

POSSIBLE FOLLOW-UP: Sequential integer IDs?

BEST RESPONSE: Audit sequence is an integer chain index; business ids are UUIDs.

Q44. Describe TargetCreate validation.

QUESTION: Describe TargetCreate validation.

SHORT ANSWER: Bounded names, timezone-aware expiry, unique capabilities from enum, scope assets pattern.

DETAILED ANSWER: Past expiry rejected; extra fields forbidden.

PROJECT-SPECIFIC IMPLEMENTATION: packages/contracts/targets.py extra=forbid.

POSSIBLE FOLLOW-UP: SQL injection via name?

BEST RESPONSE: SQLAlchemy bound parameters plus length/text reject_unbounded_text.

Q45. Case transition request?

QUESTION: Case transition request?

SHORT ANSWER: JSON {to_state, reason} with BoundedReason; must be the next enum value.

DETAILED ANSWER: Illegal skip → error, no partial write intended.

PROJECT-SPECIFIC IMPLEMENTATION: CaseTransitionRequest; packages/cases rules.

POSSIBLE FOLLOW-UP: Can two admins race?

BEST RESPONSE: Transition locks the target row then re-checks eligibility.

Q46. List GET versus POST ingest.

QUESTION: List GET versus POST ingest.

SHORT ANSWER: Ingest is POST only. Findings expose supporting_event_ids. No events list route in OpenAPI.

DETAILED ANSWER: 28-ish documented paths; telemetry/events 404.

PROJECT-SPECIFIC IMPLEMENTATION: routers/telemetry.py single POST.

POSSIBLE FOLLOW-UP: How does the ticker work then?

BEST RESPONSE: uniqueSupportingEvents(findings) in the console.

Q47. Audit verify response?

QUESTION: Audit verify response?

SHORT ANSWER: ChainVerificationView: valid, event_count, head_hash, failure_sequence, failure_reason, contract_version.

DETAILED ANSWER: GET /api/v1/audit/verify.

PROJECT-SPECIFIC IMPLEMENTATION: routers/audit.py get_verification.

POSSIBLE FOLLOW-UP: If valid is false?

BEST RESPONSE: A stored hash did not match replay — investigate DB tampering or bug; do not ignore it.

Q48. Correlation run body?

QUESTION: Correlation run body?

SHORT ANSWER: Optional window_seconds 1–3600 default 300. Admin POST. Returns run view including risk.

DETAILED ANSWER: Repeated runs update finding fingerprints instead of duplicating.

PROJECT-SPECIFIC IMPLEMENTATION: CorrelationRunRequest; 0005 unique fingerprint.

POSSIBLE FOLLOW-UP: Does it modify telemetry_events?

BEST RESPONSE: No. Original evidence is append-only.

Q49. Reasoning run body?

QUESTION: Reasoning run body?

SHORT ANSWER: window_seconds, dataset_label, proposal_kind, include_counterfactual. Admin POST assessments.

DETAILED ANSWER: GET timeline/explanation/hypotheses return stored views (explanation/hypotheses reuse assessment GET).

PROJECT-SPECIFIC IMPLEMENTATION: routers/reasoning.py.

POSSIBLE FOLLOW-UP: dataset_label effect?

BEST RESPONSE: Stored on reconstruction; UI uses synthetic_lab to show LAB ENVELOPES.

Q50. What is 409 used for?

QUESTION: What is 409 used for?

SHORT ANSWER: Eligibility failures and illegal case moves — conflict with stored authorization/state.

DETAILED ANSWER: Better than 400 for ‘business rule’.

PROJECT-SPECIFIC IMPLEMENTATION: ApiError in services.

POSSIBLE FOLLOW-UP: Is 409 in the frontend map?

BEST RESPONSE: Not specially named; message still displays via generic parse.

Cybersecurity and telemetry

Q51. Which event types are allowed?

QUESTION: Which event types are allowed?

SHORT ANSWER: process_started, process_exited, network_connection_observed in contract 1.0.0.

DETAILED ANSWER: Each has a JSON Schema under schemas/.

PROJECT-SPECIFIC IMPLEMENTATION: packages/telemetry + schemas/telemetry-*.json.

POSSIBLE FOLLOW-UP: File writes? DNS?

BEST RESPONSE: Not in v0.1 allowlist; rejected.

Q52. What is boot_id for?

QUESTION: What is boot_id for?

SHORT ANSWER: Disambiguates PID reuse across reboots; part of process identity.

DETAILED ANSWER: Joined with pid and start_ticks.

PROJECT-SPECIFIC IMPLEMENTATION: evidence-correlation.md.

POSSIBLE FOLLOW-UP: Is it Linux /proc/sys/kernel/random/boot_id?

BEST RESPONSE: The Linux agent can read procfs; pid_hash in Phase 12 adapters is explicitly not Linux boot identity. Don’t mix them.

Q53. How is redaction tested?

QUESTION: How is redaction tested?

SHORT ANSWER: Allowlist redaction on payloads so secrets in argv are not stored raw as a policy of the telemetry package.

DETAILED ANSWER: See telemetry architecture docs and unit tests.

PROJECT-SPECIFIC IMPLEMENTATION: packages/telemetry; tests around redaction.

POSSIBLE FOLLOW-UP: Is the UI XSS-safe for event strings?

BEST RESPONSE: React text interpolation, no dangerouslySetInnerHTML in the console source reviewed.

Q54. Enrollment versus monitoring.

QUESTION: Enrollment versus monitoring.

SHORT ANSWER: Enrollment stores a key. Monitoring requires ingest over time. UI copy states this.

DETAILED ANSWER: Missing ticker is not proof of zero host activity; it is absence in findings.

PROJECT-SPECIFIC IMPLEMENTATION: TelemetryOps.tsx note; Command Centre note.

POSSIBLE FOLLOW-UP: How do you show last-seen?

BEST RESPONSE: We don’t have a heartbeat API. Don’t invent one.

Q55. What stops a random key from ingesting?

QUESTION: What stops a random key from ingesting?

SHORT ANSWER: Signature must verify against the enrolled public key for that agent_id and target eligibility.

DETAILED ANSWER: Detached Base64 signature field.

PROJECT-SPECIFIC IMPLEMENTATION: ingest path; telemetry signatures.py.

POSSIBLE FOLLOW-UP: Replay the same envelope?

BEST RESPONSE: Duplicate event_id persistence returns a duplicate status rather than a second fact; demo persist shows this.

Q56. Can viewers ingest?

QUESTION: Can viewers ingest?

SHORT ANSWER: Ingest doesn’t use viewer tokens; it uses the agent signature. Viewers cannot enroll keys.

DETAILED ANSWER: Split of control-plane auth vs agent auth.

PROJECT-SPECIFIC IMPLEMENTATION: telemetry router vs agents require_admin.

POSSIBLE FOLLOW-UP: Stolen private key?

BEST RESPONSE: Until revocation/rotation, they can sign; rotation increments key_generation. No HSM in v0.1.

Q57. Process identity tuple?

QUESTION: Process identity tuple?

SHORT ANSWER: (agent_id, boot_id, pid, start_ticks). PID alone is forbidden as a join.

DETAILED ANSWER: Network binds only if exactly one covering lifetime exists.

PROJECT-SPECIFIC IMPLEMENTATION: synapse_correlation engine grouping.

POSSIBLE FOLLOW-UP: Two processes same pid different ticks?

BEST RESPONSE: Treated as different identities; pid reuse unresolved is a finding type.

Q58. Finding types names.

QUESTION: Finding types names.

SHORT ANSWER: lifecycle_exit_without_start, lifecycle_pid_reuse_unresolved, repeated_network_activity, multi_port_network_activity, rapid_activity_burst, repeated_observation_pattern.

DETAILED ANSWER: Heuristics with thresholds 5 connections, 3 ports, 8 events/2s, etc.

PROJECT-SPECIFIC IMPLEMENTATION: evidence-correlation.md table.

POSSIBLE FOLLOW-UP: Are they MITRE techniques?

BEST RESPONSE: No mapping table in repo.

Q59. Epistemic correlation versus inference.

QUESTION: Epistemic correlation versus inference.

SHORT ANSWER: Correlation links stored observations. Inference is from absence/ambiguity (exit without start).

DETAILED ANSWER: UI must not promote inference to observed.

PROJECT-SPECIFIC IMPLEMENTATION: FindingView epistemic_status; Timeline classification.

POSSIBLE FOLLOW-UP: Is inferred false?

BEST RESPONSE: Not necessarily; it is not directly observed. We keep the label.

Q60. Risk contribution versus confidence.

QUESTION: Risk contribution versus confidence.

SHORT ANSWER: Contribution feeds the capped sum. Confidence/evidence_strength are completeness-style fields, different axis from severity.

DETAILED ANSWER: Severity bands 0–19 info … 70–100 high from score.

PROJECT-SPECIFIC IMPLEMENTATION: scoring.py header comments.

POSSIBLE FOLLOW-UP: If confidence is 0.8 is it 80% true?

BEST RESPONSE: No. Engine score only.

Q61. What is content_class / basis on envelopes?

QUESTION: What is content_class / basis on envelopes?

SHORT ANSWER: Contract fields on the signed envelope (schema 1.0.0) describing observation basis.

DETAILED ANSWER: Must be in canonical JSON so the signature covers them.

PROJECT-SPECIFIC IMPLEMENTATION: telemetry-envelope schema.

POSSIBLE FOLLOW-UP: Can you strip them after verify?

BEST RESPONSE: Stored evidence should remain as verified; don’t describe a strip API that isn’t there.

Q62. Laboratory envelopes vs genuine /proc.

QUESTION: Laboratory envelopes vs genuine /proc.

SHORT ANSWER: Lab envelopes are constructed JSON signed and ingested for demos. /proc would come from packages/agent on a real host.

DETAILED ANSWER: dataset_label synthetic_lab triggers LAB ENVELOPES in HUD.

PROJECT-SPECIFIC IMPLEMENTATION: provenance.ts; agent package.

POSSIBLE FOLLOW-UP: Did you submit /proc in the marked live test?

BEST RESPONSE: The documented seed did not; say laboratory envelopes honestly.

Digital signatures and audit integrity

Q63. How does Ed25519 work here?

QUESTION: How does Ed25519 work here?

SHORT ANSWER: 32-byte public, 64-byte signature over SHA-512 internally as specified by Ed25519; we use cryptography.io, not a homemade curve.

DETAILED ANSWER: We sign domain||canonical JSON, not raw payload only.

PROJECT-SPECIFIC IMPLEMENTATION: docs/architecture/telemetry-signing.md; signatures.py.

POSSIBLE FOLLOW-UP: Does Ed25519 encrypt?

BEST RESPONSE: No. It authenticates. Confidentiality is not the ingest goal.

Q64. Why a domain string?

QUESTION: Why a domain string?

SHORT ANSWER: Prevents this signature being reused as a signature for another protocol.

DETAILED ANSWER: Fixed ASCII synapse-crs/telemetry-signature/1 plus newline.

PROJECT-SPECIFIC IMPLEMENTATION: telemetry-signing.md What is signed.

POSSIBLE FOLLOW-UP: If we change canonicalization?

BEST RESPONSE: Need a new domain version; old signatures won't verify.

Q65. Canonical JSON rules?

QUESTION: Canonical JSON rules?

SHORT ANSWER: UTF-8, sorted keys, compact separators, ensure_ascii false, no floats.

DETAILED ANSWER: Same family as audit hashing.

PROJECT-SPECIFIC IMPLEMENTATION: chain.normalize_payload; telemetry canonicalization.

POSSIBLE FOLLOW-UP: List order?

BEST RESPONSE: Preserved. Dict insertion order is not relied on because keys are sorted.

Q66. Audit hash formula?

QUESTION: Audit hash formula?

SHORT ANSWER: h = SHA256(JSON of {actor, ids, type, payload, previous_event_hash, sequence, timestamp}).

DETAILED ANSWER: Genesis previous hash is 64 zeros. Timestamps fixed-width microseconds Z.

PROJECT-SPECIFIC IMPLEMENTATION: packages/audit/src/synapse_audit/chain.py compute_event_hash.

POSSIBLE FOLLOW-UP: MD5?

BEST RESPONSE: No, SHA-256.

Q67. What if an attacker UPDATEs an audit row as synapse_app?

QUESTION: What if an attacker UPDATEs an audit row as synapse_app?

SHORT ANSWER: Trigger should reject UPDATE/DELETE/TRUNCATE for that role.

DETAILED ANSWER: DB owner can still disable triggers; verification then depends on whether hashes still match a replay.

PROJECT-SPECIFIC IMPLEMENTATION: docs/decisions/0003-audit-append-only.md; claims.md audit limitation.

POSSIBLE FOLLOW-UP: Is this legally admissible logging?

BEST RESPONSE: Not claimed. Tamper-evidence for the app role only.

Q68. What does verify do?

QUESTION: What does verify do?

SHORT ANSWER: Replays hashes in sequence order and compares stored head.

DETAILED ANSWER: Returns valid boolean and failure_sequence if any.

PROJECT-SPECIFIC IMPLEMENTATION: audit.py get_verification → verify_stored_chain.

POSSIBLE FOLLOW-UP: Empty database?

BEST RESPONSE: Zero events, genesis head; treat as valid empty chain (describe what the function returns if asked — don't guess a number).

Q69. Are tokens hashed at rest in .env?

QUESTION: Are tokens hashed at rest in .env?

SHORT ANSWER: No. SYNAPSE_AUTH_TOKENS is plaintext in the environment. HMAC is only for compare at auth time.

DETAILED ANSWER: Appropriate for local lab, wrong for a public SaaS (ADR 0002).

PROJECT-SPECIFIC IMPLEMENTATION: config.py parse_auth_tokens.

POSSIBLE FOLLOW-UP: Could you store SHA-256 of tokens?

BEST RESPONSE: Yes as future work; not implemented. Auth currently needs the raw token map.

Q70. TLS?

QUESTION: TLS?

SHORT ANSWER: Local demo is HTTP on loopback. Production HTTPS is an ops concern, not a feature toggle in code.

DETAILED ANSWER: Do not claim mTLS agents.

PROJECT-SPECIFIC IMPLEMENTATION: No TLS terminator in infra/docker-compose.yml.

POSSIBLE FOLLOW-UP: Is loopback enough?

BEST RESPONSE: For a viva on one machine yes; not for a shared network.

Neuro-symbolic reasoning and algorithms

Q71. What does neuro-symbolic mean here?

QUESTION: What does neuro-symbolic mean here?

SHORT ANSWER: Isolation Forest unusualness plus inspectable symbolic rules combined by hybrid selection and an explanation object.

DETAILED ANSWER: Not a large language model. Not a graph neural net.

PROJECT-SPECIFIC IMPLEMENTATION: packages/reasoning hybrid.py, neural.py, symbolic.py.

POSSIBLE FOLLOW-UP: Where is the neural net architecture?

BEST RESPONSE: sklearn-style Isolation Forest baseline on synthetic features, not a deep net.

Q72. Why Isolation Forest?

QUESTION: Why Isolation Forest?

SHORT ANSWER: Unsupervised unusualness on numeric features; we explicitly do not treat the score as P(attack).

DETAILED ANSWER: Trained in the synthetic research path; limitations string on NeuralView.

PROJECT-SPECIFIC IMPLEMENTATION: neural.py; assessment.neural.limitations in UI.

POSSIBLE FOLLOW-UP: Why not a supervised CNN on CIC-IDS?

BEST RESPONSE: CIC-IDS is not bundled; Phase 11 is a separate fail-closed flow track. Mixing would be dishonest.

Q73. What is unusualness versus probability?

QUESTION: What is unusualness versus probability?

SHORT ANSWER: A score compared to a threshold flagging unusual=true/false. No calibration to base rates.

DETAILED ANSWER: Reasoning theatre copy forbids probability language.

PROJECT-SPECIFIC IMPLEMENTATION: ReasoningEngine.tsx neural stage text.

POSSIBLE FOLLOW-UP: Can I apply softmax?

BEST RESPONSE: Not in code; I would refuse to present it as a calibrated posterior.

Q74. How is H4 selected?

QUESTION: How is H4 selected?

SHORT ANSWER: If reconstruction_completeness < 0.4 or fewer than two observations, H4 support at least 0.85 and H3 capped. Ties prefer H4.

DETAILED ANSWER: Rationale: sparse evidence is insufficient, not malicious.

PROJECT-SPECIFIC IMPLEMENTATION: hybrid.py _hypotheses and _select_id.

POSSIBLE FOLLOW-UP: Is that Bayesian?

BEST RESPONSE: No. It is a conservative max-support rule.

Q75. Name H1 H2 H3 H4.

QUESTION: Name H1 H2 H3 H4.

SHORT ANSWER: normal; unusual_legitimate; suspicious_pattern; insufficient_evidence.

DETAILED ANSWER: Statuses selected/viable/rejected from support thresholds.

PROJECT-SPECIFIC IMPLEMENTATION: HYPOTHESES tuple in hybrid.py.

POSSIBLE FOLLOW-UP: Can two be selected?

BEST RESPONSE: No. _select picks one id.

Q76. Feature ablation / counterfactual?

QUESTION: Feature ablation / counterfactual?

SHORT ANSWER: Optional include_counterfactual. Limitations say ablation is not causal inference.

DETAILED ANSWER: UI ablation panel quotes stored description.

PROJECT-SPECIFIC IMPLEMENTATION: counterfactual.py; claims and research docs.

POSSIBLE FOLLOW-UP: Did removing a feature prove the cause?

BEST RESPONSE: No. That is a documented forbidden claim.

Q77. Event-count prediction?

QUESTION: Event-count prediction?

SHORT ANSWER: predict_event_volume compared to persistence; Phase 10 reported persistence advantage False on synthetic.

DETAILED ANSWER: Not a demonstrated forecasting product.

PROJECT-SPECIFIC IMPLEMENTATION: prediction.py; benchmark-results.md.

POSSIBLE FOLLOW-UP: Stock price analogy?

BEST RESPONSE: Avoid. Stick to event counts in a window.

Q78. Z3 SAT meaning?

QUESTION: Z3 SAT meaning?

SHORT ANSWER: sat: policy Booleans consistent; unsat: not; unknown: timeout 2s.

DETAILED ANSWER: Explanation strings in verification.py state SAT is not real-world safety.

PROJECT-SPECIFIC IMPLEMENTATION: verify_proposal.

POSSIBLE FOLLOW-UP: Which theory?

BEST RESPONSE: Boolean SAT via Z3; not nonlinear real arithmetic over kernel memory.

Q79. Write the policy in words.

QUESTION: Write the policy in words.

SHORT ANSWER: Must not execute, must not be destructive, must be authorized and in scope, identity certain, evidence present, impactful implies human approval, mitigation kind forbidden.

DETAILED ANSWER: Impactful kinds include recommend_isolate_lab_target and execute_host_mitigation; the latter sets destructive/mitigation flags.

PROJECT-SPECIFIC IMPLEMENTATION: propose() and CONSTRAINTS.

POSSIBLE FOLLOW-UP: If SAT on recommend_isolate… did we isolate?

BEST RESPONSE: No. executed is forced false. Recommendation record only.

Q80. Risk formula again with numbers.

QUESTION: Risk formula again with numbers.

SHORT ANSWER: min(100, sum of capped groups). Example 15+8+20=43.

DETAILED ANSWER: Caps prevent one noisy type from saturating attention.

PROJECT-SPECIFIC IMPLEMENTATION: scoring.py FORMULA and ROLLUP_CAPS.

POSSIBLE FOLLOW-UP: Why cap pid reuse at 50?

BEST RESPONSE: Heuristic documented in scoring.py, not fitted on enterprise data.

Q81. Reconstruction completeness?

QUESTION: Reconstruction completeness?

SHORT ANSWER: A 0–1 quality field on the reconstruction; low values push H4.

DETAILED ANSWER: Also identity_verification, lifecycle_completeness, etc.

PROJECT-SPECIFIC IMPLEMENTATION: EvidenceQualityView; reconstruction.py.

POSSIBLE FOLLOW-UP: Is 1.0 perfect knowledge of the host?

BEST RESPONSE: No. It is completeness of the stored reconstruction object.

Q82. Does the system do causal inference?

QUESTION: Does the system do causal inference?

SHORT ANSWER: No. Counterfactual module is labelled non-causal.

DETAILED ANSWER: Do not cite Pearl and claim implementation.

PROJECT-SPECIFIC IMPLEMENTATION: docs/architecture/counterfactual-reasoning.md limitations.

POSSIBLE FOLLOW-UP: Then why show ablation?

BEST RESPONSE: Sensitivity of the selected hypothesis to features, with a limitation sentence beside it.

Testing, datasets and research methodology

Q83. How is the Python suite run?

QUESTION: How is the Python suite run?

SHORT ANSWER: make test → pytest against synapse_test created from MIGRATION_DATABASE_URL, not truncating synapse.

DETAILED ANSWER: 430 tests passed in one development run; quote only runs you performed.

PROJECT-SPECIFIC IMPLEMENTATION: tests/integration/conftest.py.

POSSIBLE FOLLOW-UP: Can CI passwords be reused locally?

BEST RESPONSE: docs/development/workflow.md: CI fixture must not be reused elsewhere.

Q84. What does Phase 10 measure?

QUESTION: What does Phase 10 measure?

SHORT ANSWER: Hybrid vs correlation vs neural vs symbolic on 12 synthetic reconstructions.

DETAILED ANSWER: Hybrid and symbolic 3-class accuracy 1.0 synthetic; neural-only 0.3333; CIC-IDS false.

PROJECT-SPECIFIC IMPLEMENTATION: docs/research/benchmark-results.md.

POSSIBLE FOLLOW-UP: Why is 1.0 not impressive?

BEST RESPONSE: n=12 constructed labels; hybrid can match the labelling rules. I say that before the number.

Q85. Phase 11?

QUESTION: Phase 11?

SHORT ANSWER: Separate flow track. Fails closed without SYNAPSE_CICIDS_CSV. Phase 9 hybrid not comparable on flows.

DETAILED ANSWER: make research-eval-phase11.

PROJECT-SPECIFIC IMPLEMENTATION: packages/validation/phase11.

POSSIBLE FOLLOW-UP: Did you evaluate CIC-IDS?

BEST RESPONSE: Not unless that CSV was present. Default report: not present.

Q86. Phase 12 and 13?

QUESTION: Phase 12 and 13?

SHORT ANSWER: Endpoint adapters and bounded acquisition. DMBD trees ~969MB not auto-downloaded. Label JSON ≠ trees. scientific_completion false by default.

DETAILED ANSWER: Sample labels are not event-level truth; pid_hash ≠ Linux boot_id.

PROJECT-SPECIFIC IMPLEMENTATION: docs/research/phase-12-*.md; phase-13/.

POSSIBLE FOLLOW-UP: Could you download DMBD in the viva?

BEST RESPONSE: Not mid-defence; 8 MiB bound refused trees_0.json. Explain fail-closed.

Q87. Why fail-closed on missing data?

QUESTION: Why fail-closed on missing data?

SHORT ANSWER: To stop fake leaderboard numbers.

DETAILED ANSWER: validation runners return blockers instead of fixture fallback.

PROJECT-SPECIFIC IMPLEMENTATION: phase11/12/13 run.py messages.

POSSIBLE FOLLOW-UP: Isn't that just missing work?

BEST RESPONSE: It is incomplete science plus complete engineering of the refusal path — both are true.

Q88. Frontend tests you can name.

QUESTION: Frontend tests you can name.

SHORT ANSWER: research.test.ts asserts independent=false; client nested 401; uniqueSupportingEvents; provenance laboratory note; H4 not probability.

DETAILED ANSWER: npm test.

PROJECT-SPECIFIC IMPLEMENTATION: frontend/src/**/*.test.ts.

POSSIBLE FOLLOW-UP: Coverage percent?

BEST RESPONSE: No coverage gate in package.json; don't invent 90%.

Q89. What is dataset_label synthetic_lab?

QUESTION: What is dataset_label synthetic_lab?

SHORT ANSWER: A field on reconstructions/assessments marking the synthetic laboratory track.

DETAILED ANSWER: UI lab banner keys off this string.

PROJECT-SPECIFIC IMPLEMENTATION: ReasoningRunRequest; provenanceFlag.

POSSIBLE FOLLOW-UP: If I ingest real /proc with that label?

BEST RESPONSE: The label would be a lie. Operators should set a truthful dataset_label.

Q90. Reproducibility commands?

QUESTION: Reproducibility commands?

SHORT ANSWER: make research-eval, research-eval-phase11/12/13; docs/guides/reproduce-*.md.

DETAILED ANSWER: They do not download corpora.

PROJECT-SPECIFIC IMPLEMENTATION: Makefile; docs/guides.

POSSIBLE FOLLOW-UP: One-click reproduce CIC-IDS?

BEST RESPONSE: No. Operator must supply the CSV path.

Limitations, deployment and future scope

Q91. Does the platform perform in-memory hotpatching?

QUESTION: Does the platform perform in-memory hotpatching?

SHORT ANSWER: No. Not in routers, packages, or tests.

DETAILED ANSWER: claims.md explicit non-goal. Name is not evidence.

PROJECT-SPECIFIC IMPLEMENTATION: grep would find no patch executor.

POSSIBLE FOLLOW-UP: Will you add it next semester?

BEST RESPONSE: Only with a narrow catalogue, tests, and rewritten claims — not as a hidden flag.

Q92. Can it automatically block an attacker?

QUESTION: Can it automatically block an attacker?

SHORT ANSWER: No. No firewall, isolate, or kill API. Z3 forbids mitigation_kind.

DETAILED ANSWER: Defence Lab copy in DefenceLab.tsx.

PROJECT-SPECIFIC IMPLEMENTATION: verification.py Not(mitigation).

POSSIBLE FOLLOW-UP: What about recommend_isolate_lab_target?

BEST RESPONSE: A recommendation record, executed false, still not an iptables change.

Q93. Biggest technical limitation?

QUESTION: Biggest technical limitation?

SHORT ANSWER: Scientific generalization (tiny synthetic set, missing independent corpora) plus no execution by design.

DETAILED ANSWER: Pick both if asked for one: generalization for research viva; no-exec if they want product.

PROJECT-SPECIFIC IMPLEMENTATION: benchmark-results.md; 0006-no-execution.md.

POSSIBLE FOLLOW-UP: Is the code quality the limitation?

BEST RESPONSE: There is pytest/ruff/CI; I won’t hide science gaps behind ‘needs refactor’.

Q94. How do you deploy?

QUESTION: How do you deploy?

SHORT ANSWER: Compose: postgres, migrate, api. Or host uvicorn. Console is a static Vite build or npm run dev.

DETAILED ANSWER: No Kubernetes manifests as first-class product.

PROJECT-SPECIFIC IMPLEMENTATION: infra/docker-compose.yml; infra/docker/api.Dockerfile.

POSSIBLE FOLLOW-UP: Multi-region?

BEST RESPONSE: Not designed.

Q95. Secrets management?

QUESTION: Secrets management?

SHORT ANSWER: Gitignored .env; no tokens in PDFs or git. Production would need a secret manager — not implemented.

DETAILED ANSWER: Frontend never prints full bearer.

PROJECT-SPECIFIC IMPLEMENTATION: .gitignore .env; maskToken.

POSSIBLE FOLLOW-UP: Are demo tokens in the repo?

BEST RESPONSE: Demo UI uses fixtures without production SYNAPSE_AUTH_TOKENS. Do not commit real tokens.

Q96. What if the database goes down?

QUESTION: What if the database goes down?

SHORT ANSWER: Ready 503; console DB lamp unknown/off; writes fail. Health may still be ok.

DETAILED ANSWER: No offline write-ahead in the API.

PROJECT-SPECIFIC IMPLEMENTATION: health.py ready(); Shell lamps.

POSSIBLE FOLLOW-UP: Automatic failover?

BEST RESPONSE: Not implemented.

Q97. OIDC future?

QUESTION: OIDC future?

SHORT ANSWER: ADR 0002 says tokens can sit behind OIDC later. Not built.

DETAILED ANSWER: Same role checks could remain.

PROJECT-SPECIFIC IMPLEMENTATION: docs/decisions/0002-bearer-tokens.md.

POSSIBLE FOLLOW-UP: Why not JWT now?

BEST RESPONSE: Local research deployment; fewer moving parts.

Q98. How would you add real /proc telemetry honestly?

QUESTION: How would you add real /proc telemetry honestly?

SHORT ANSWER: Run packages/agent against an owned lab host, enroll its key, ingest, and do not mark dataset_label synthetic_lab.

DETAILED ANSWER: Keep demo fixtures isolated.

PROJECT-SPECIFIC IMPLEMENTATION: docs/guides/linux-agent-setup.md.

POSSIBLE FOLLOW-UP: Mix /proc and synthetic in one graph?

BEST RESPONSE: Architecture forbids mixing validation corpora into telemetry_events; I would also keep labels strict.

Q99. What must you not say in the viva?

QUESTION: What must you not say in the viva?

SHORT ANSWER: Zero-day detection; 99% accuracy; SAT means safe; hotpatch live; GET all telemetry; CIC-IDS success without CSV; silent demo substitution.

DETAILED ANSWER: Handbook speaking rules.

PROJECT-SPECIFIC IMPLEMENTATION: DEMO_RUNBOOK claims section; claims.md.

POSSIBLE FOLLOW-UP: If the examiner wants a wow demo?

BEST RESPONSE: Show H4 and audit verify — honesty is the wow.

Q100. What is your personal contribution story (template)?

QUESTION: What is your personal contribution story (template)?

SHORT ANSWER: Speak only to work you did: e.g. console workspaces, live integration, documentation package. Do not claim authorship of every package if you did not write it.

DETAILED ANSWER: Adjust this answer to the real student history. The repo is a team/research codebase.

PROJECT-SPECIFIC IMPLEMENTATION: UNVERIFIED per student — fill before viva.

POSSIBLE FOLLOW-UP: Did you write the Isolation Forest?

BEST RESPONSE: If you did not, say you integrated/explained the existing package. Fabricating authorship is worse than incomplete science.